

Contents

Raíces del Código	3
Fundamentos de Java a través de la gestión de viveros forestales	3
I. Prólogo.....	4
II. Introducción	5
Ejercicio 0: Hola Bosque	6
Ejercicio 1: Área de Plantación	7
Ejercicio 2: Conteo de Brotes	8
Ejercicio 3: Control de Altura	9
Ejercicio 4: Alerta de Sequía	10
Ejercicio 5: Registro de Crecimiento	11
Ejercicio 6: Ficha del Vivero	12
Ejercicio 7: Inventario de Semillas	13
RETO: Mi primer Hola Bosque	14
III. Ejecución	16

Raíces del Código

Java es un lenguaje de programación **orientado a objetos (POO)** que se usa para crear desde aplicaciones de escritorio, móviles (como aplicaciones Android) hasta servidores y sistemas empresariales. Su filosofía principal es “**escribe una vez, corre en cualquier lugar**” porque funciona sobre la **Java Virtual Machine (JVM)**, que traduce el código Java a un lenguaje que cualquier sistema operativo puede ejecutar.

Fundamentos de Java a través de la gestión de viveros forestales

Construye un ecosistema digital de gestión forestal mientras descubres conceptos avanzados de Java como Clases, Herencia, Encapsulamiento y Polimorfismo

Instrucciones Generales

- ✓ **Versión:** Java 17+ (OpenJDK).
- ✓ **Estructura:** Una clase por archivo (sin package).
- ✓ **Linter Obligatorio:** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ **Comentarios:** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ **IDE:** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ **Compilación:** Vía terminal: `javac Clase.java` y `java Clase`.

Entrega y Evaluación

- Entrega en repositorio Git.

I. Prólogo

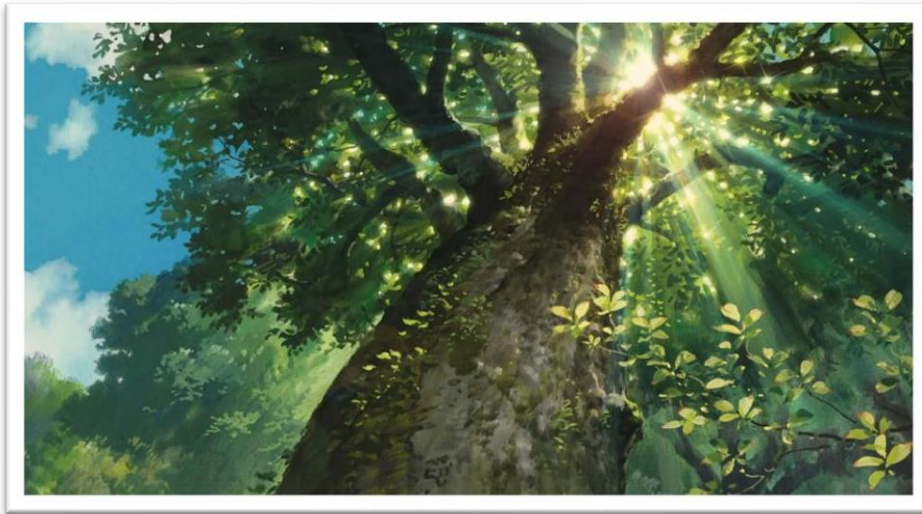
En los viveros forestales, cada semilla representa el futuro de un ecosistema. El seguimiento preciso de la germinación y el crecimiento es vital para asegurar que los bosques del mañana sean fuertes.

Java, al igual que un bosque de secuoyas, es conocido por su estructura, su altura y su capacidad para perdurar en el tiempo. A través de este proyecto, aprenderás que programar, al igual que la silvicultura, requiere paciencia, estructura y una atención meticulosa a los detalles.



II. Introducción

*¡Te damos la bienvenida a **Raíces del Código!***



Trabajarás con ejercicios prácticos que introducen conceptos de Java en un contexto de gestión forestal.

IMPORTANTE: En cada ejercicio, debes entregar una **Clase** que contenga el método estático solicitado.

- ✓ La gestión de entrada y salida (**`System.out`** y **`Scanner`**) debe ocurrir dentro del método.
-

Ejercicio 0: Hola Bosque

- **Directorio:** ex00/
- **Archivo:** FtHelloForest.java
- **Método:** ftHelloForest()
- **Funciones Autorizadas:** System.out.println

Escribe un método que muestre un mensaje de bienvenida al sistema de gestión forestal.
Crear un main que lo ejecute y demuestre que funciona.

Salida Esperada

```
Welcome to the Forest Management System!
```

Ejercicio 1: Área de Plantación

- **Directorio:** ex01/
- **Archivo:** FtPlantingArea.java
- **Método:** ftPlantingArea()
- **Funciones Autorizadas:** Scanner, System.out.print/ln, Integer.parseInt

Solicita la longitud y el ancho (en metros) de una sección de reforestación y muestra el área total. **Crear un main** que lo ejecute y demuestre que funciona.

Salida Esperada

```
Enter length: 10
Enter width: 20
Total planting area: 200
```

Ejercicio 2: Conteo de Brotes

- **Directorio:** ex02/
- **Archivo:** FtSproutCount.java
- **Método:** ftSproutCount()
- **Funciones Autorizadas:** Scanner, System.out.print/ln

Un guardabosques cuenta los brotes nuevos en 3 parcelas distintas. Solicita los 3 valores y muestra la suma total. **Crear un main** que lo ejecute y demuestre que funciona.

Salida Esperada

```
Plot 1 sprouts: 50
Plot 2 sprouts: 30
Plot 3 sprouts: 20
Total sprouts counted: 100
```

Ejercicio 3: Control de Altura

- **Directorio:** ex03/
- **Archivo:** FtTreeHeight.java
- **Método:** ftTreeHeight()
- **Funciones Autorizadas:** Scanner, System.out.print/ln

Solicita la altura de un ejemplar en centímetros.

Si mide más de 100 cm, indica que está listo para ser trasplantado al bosque y si no muestra "Not ready for transplanting!".

Crear un main que lo ejecute y demuestre que funciona.

Salida Esperada

```
Enter tree height (cm): 120
Ready for transplanting!
```

Ejercicio 4: Alerta de Sequía

- **Directorio:** ex04/
- **Archivo:** FtDroughtAlert.java
- **Método:** ftDroughtAlert()
- **Funciones Autorizadas:** Scanner, System.out.print/ln

Solicita los litros de lluvia caídos esta semana. Si es menor a 15 litros, ¡muestra "Activate irrigation system!", de lo contrario "Moisture levels optimal". **Crear un main** que lo ejecute y demuestre que funciona.

Salida Esperada

```
Rainfall (liters): 10
Activate irrigation system!
```

Ejercicio 5: Registro de Crecimiento

- **Directorio:** ex05/
- **Archivo:** FtGrowthLog.java
- **Métodos:** ftGrowthLogIterative(), ftGrowthLogRecursive(int n) y ftGrowthLogRecursiveHelper(int n, int day)
- **Funciones Autorizadas:** Scanner, System.out.print/ln

Ambos métodos deben imprimir el progreso día a día hasta llegar al objetivo dado por el usuario. **Crear un main** que lo ejecute y demuestre que funciona.

Salida Esperada

```
Target day: 3
Day 1: Growing...
Day 2: Growing...
Day 3: Growing...
Target reached!
```

Ejercicio 6: Ficha del Vivero

- **Directorio:** ex06/
- **Archivo:** FtNurserySummary.java
- **Método:** ftNurserySummary()
- **Funciones Autorizadas:** Scanner, System.out.print/ln, nextLine().

Solicita el nombre del vivero y la especie principal de árbol, mostrando un resumen formateado.

Crear un main que lo ejecute y demuestre que funciona.

Salida Esperada

```
Enter nursery name: Valle Verde
Enter main species: Roble
Nursery: Valle Verde | Species: Roble | Status: Active
```

Ejercicio 7: Inventario de Semillas

- **Directorio:** ex07/
- **Archivo:** FtForestSeeds.java
- **Método:** ftForestSeeds(String type, int qty, String unit)
- **Funciones Autorizadas:** System.out.print/ln, String.substring, String.toUpperCase/LowerCase

El método recibe parámetros y debe validar la unidad ("kilograms", "seeds", "bags").

- Si la unidad no es válida, mostrar "Unknown unit".
- Formatear el tipo de semilla para que empiece en mayúscula.
- Usar switch.
- **Crear un main** que lo ejecute y demuestre que funciona.

Salida Esperada

```
Pine: 50 kilograms in stock.  
Oak: 200 seeds ready for planting.  
Fir: 10 bags of seeds available.  
Unknown unit.
```

RETO: Mi primer Hola Bosque

Spring Boot es un framework de Java para crear aplicaciones (sobre todo backend/APIs) de forma rápida, sin tener que configurar todo a mano. **Framework** es un "esqueleto" de código ya hecho que te da herramientas y estructura para no reinventar la rueda cada vez. En vez de escribir desde cero cómo manejar peticiones HTTP, conexiones a bases de datos, seguridad, etc., el framework ya trae eso resuelto — tú solo lo usas y personalizas.

Directorio: reto/ - Tener en cuenta que esta carpeta va fuera de todos los modulos, porque se volverá a usar.

Objetivo

- ✓ Ver la diferencia entre imprimir por consola (**`System.out.println`**) e imprimir en una página web, sin entender aún el "por qué" — solo sentir el "wow, esto también es Java".
- ✓ No necesitas entender **`@RestController`** ni **`@GetMapping`** todavía — solo necesitas **verlo funcionar**.
- ✓ Spring Boot genera un proyecto completo desde el inicio, con carpetas y archivos que en tu Java puro (los `ex00`, `ex01`...) no existen.

Pasos

1. Ve a start.spring.io (el generador oficial de proyectos Spring Boot)
2. Configura:
 - **Project:** Maven
 - **Language:** Java
 - **Spring Boot:** elige la versión **4.1.0**
 - **Dependencies (esto es importante, no lo dejes vacío):** Spring Web

Project Metadata

- **Group:** `com.forestapp` (o lo que quieras, es como el "paquete" de tu empresa/proyecto)
 - **Artifact:** `hola-bosque`
 - **Package name:** se va a autocompletar solo (algo como `com.forestapp.holabosque`)
 - **Packaging:** Jar (déjalo así, es lo estándar)
 - **Java:** `java -version` (En tu ordenador)
3. Dale a **"Generate"**, descomprime el `.zip`
 4. Abre el proyecto (en VSCode o el editor que uses) y busca el archivo **`DemoApplication.java`** (o como se llame tu proyecto)
 5. Justo antes de la clase, agrega esta línea mágica (no te preocupes por entenderla aún, solo cópiala): **`@RestController`**

6. Dentro de la clase, agrega este método:

```
@GetMapping("/hola-bosque")
public String holaBosque() {
    return "Bienvenido al sistema de gestión forestal";
}
```

7. Ejecuta la aplicación (botón **"Run"** o **mvn spring-boot:run**)

8. Abre tu navegador y entra a: **<http://localhost:8080/hola-bosque>**

Spring es el framework grande y "papá" de todo esto. Es muy potente pero antiguamente requería muchísima configuración manual (archivos XML larguísimos, etc.) Spring Boot nació para simplificar eso. Es Spring, pero con configuración automática ("convención sobre configuración"). Básicamente te deja arrancar un proyecto funcionando en minutos, en vez de horas configurando.

III. Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando `javac` te dirá "**archivo no encontrado**" y te entrarán ganas de tirar el PC por la ventana.

- ✓ `cd Desktop/proyectos_java/ex00`
- ✓ `javac FtHelloForest.java` -> **La Compilación (El traductor)**
- ✓ `java FtHelloForest` -> **La Ejecución (El lanzamiento)**